

秩奇异值与条件数实验

目标与证据

本实验在CPU上对小矩阵分解、重构、截断，再解一个右端轻微变化的线性系统。请同时提交手算、程序结果和解释：矩阵重构误差测什么，条件数界在哪个扰动方向达到，小能量方向为什么仍可能是分类所需信息。实验不训练神经网络，不下载数据，也不报告真实任务性能。

先阅读正文第一至十节。唯一正式先修是009；本实验不要求导数、自动微分、最小二乘或GPU。建议按A至H完成，先预测后运行，再对照answers。预计需要纸笔、能运行NumPy的Python环境和约一至两小时，时间取决于对矩阵运算的熟悉程度。

环境与文件

推荐Python 3.12、NumPy 2.3.5；environment.yml另外列出JupyterLab与ipykernel。可在本目录使用下面命令创建学习环境。此命令是供读者使用的安装说明，本次作者没有实际新建Anaconda/conda环境，也没有做Windows或macOS安装测试。

```
conda env create -f environment.yml
conda activate dl-unit-013
```

experiment.py含可导入的函数与完整脚本，experiment.ipynb分成逐步的教学单元格，test_experiment.py提供独立核验。data/config.json记录主矩阵与扰动参数，data/README.md给出含义。随附outputs保存本次脚本结果；Notebook内部保留实际执行输出，重跑时会另外生成notebook_outputs，这个重复目录不随资料发布。

核心命令在013目录运行：

```
python experiment.py --output outputs
python test_experiment.py
jupyter lab experiment.ipynb
```

前两条已在新Python进程实际执行；第三条是读者启动浏览器界面的命令，本次未测试Jupyter浏览器UI。提供的Notebook使用新进程中的真实IPython InProcessKernel从空状态顺序执行。执行环境限制socket，未验证跨进程内核传输。不要把Notebook已有输出当作你本地已执行的证据：应重启内核，再从第一格运行全部。

任务A 先算再看返回值

把 $W = \begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 0 \end{bmatrix}$ 写在纸上，逐项算 WW^T ，验证 $u_1 = (1, 1)^T / \sqrt{2}$ 、 $u_2 = (1, -1)^T / \sqrt{2}$ 对应16和4两个特征值。再用 $v_i = W^T u_i / \sigma_i$ 求右奇异向量。写清为什么奇异值是4和2，不是16和4。

运行Notebook的“约简SVD”格。预期 $U.shape=(2,2)$ 、 $s.shape=(2,)$ 、 $Vt.shape=(2,3)$ 。用 $U^T U$ 与 $Vt Vt^T$ 核对正交性，用 $(U*s)@Vt$ 核对重构。这里 s 是按列缩放 U 的一维数组；若不熟悉广播，先改成 $U@np.diag(s)@Vt$ 。

检查点：允许 u_i 、 v_i 同时变号，不要求与纸上每个元素相同。记录最大绝对重构误差，并解释它为什么应很小却不一定精确为0。若你的程序通过全等号比较失败，不要立刻认定SVD错了。

任务B 截断与两种误差

对 $k=0, 1, 2$ 调用`rank_k`。 $k=0$ 得到全零矩阵； $k=1$ 应约为 $[[2,2,0],[2,2,0]]$ ； $k=2$ 恢复原矩阵。每个结果仍是 2×3 ， k 不是输出数组列数，而是允许保留的方向数。

分别核验Frobenius误差 $\sqrt{20}$ 、2、0以及谱误差4、2、0。前者测所有元素误差的平方和，后者测最坏单位输入的输误差。最后一项在浮点计算中按接近0检查。填写`rank_errors.csv`时，`observed`值和尾部公式应在容差内相等。

```
from experiment import rank_k, frobenius
import numpy as np
W = np.array([[3., 1., 0.], [1., 3., 0.]])
for k in range(3):
    print(k, frobenius(W - rank_k(W, k)))
```

检查点：80%保留平方能量不等于80%“准确率”，也不等于20%相对范数误差。自行计算 $2/\sqrt{20}$ ，写出两种百分比的分母。再用输入 $x=(1,-1)/\sqrt{2}$ 验证谱误差2可以真正达到。

任务C 证明任务信息可以与能量排序相反

固定 $H=\text{diag}(10,0.1)$ ，输入两行 $(1,1)$ 、 $(1,-1)$ ，标签分别为+和-。先用完整 H ，再用`rank_k(H,1)`计算输出。观察第二列从 ± 0.1 变为0，而第一列都为10。

报告相对Frobenius误差，以及两类输出是否还能区分。这里不需要训练分类器：如果输出已经相同，任何只看输出的确定性预测器都会给出同一结果。请说明为何这个反例足以否定“保留99%能量必定保留标签信息”，却不能证明所有低秩压缩都无效。

任务D 让一个小扰动走到弱方向

`data/config.json`的 δ 固定为 10^{-6} ， ϵ 取 1 、 10^{-2} 、 10^{-4} 、 10^{-6} 。每次构造 $W\epsilon=\text{diag}(1,\epsilon)$ ，解 $xW\epsilon=(1,0)$ ，再解 $x'W\epsilon=(1,\delta)$ 。应使用`solve_row`，它通过转置调用线性求解器，不能把行方程直接交给列方程接口。

逐行记录 κ_2 、相对右端变化、相对解变化、放大倍数和对新右端的残差。预期放大倍数为1、100、10000、1000000。 $\epsilon=\delta=10^{-6}$ 时， x' 应为 $(1,1)$ 。解释“输入扰动很小”指右端相对变化 10^{-6} ，不是说参数解也只改了 10^{-6} 。

追加对照：将 δb 改成 $(\delta, 0)$ ，其余不变。观察同一病态矩阵的安全方向放大倍数为1。没有这个对照，很容易把条件数误解成每个扰动的固定放大系数。

任务E 检查残差不能代替解误差

仍令 $\varepsilon=10^{-6}$ 、原右端 $b=(1,0)$ 。取候选解 $\hat{x}=(1,1)$ 。手算 $\hat{x}^T W \varepsilon - b = (0, 10^{-6})$ ，残差范数 10^{-6} ；真实解 $x=(1,0)$ ，相对解误差为1。

再看脚本报告中的perturbed_residual，它相对于新右端 $b+\delta b$ 计算，理论上应为0，浮点下应接近0。请在报告中区分原右端残差与新右端残差，不能把它们混写成同一列。这个任务说明小残差是必要诊断，但病态时不是解可靠性的充分证据。

任务F 把数值秩的阈值写出来

对 $\text{diag}(1, 10^{-8})$ ，分别调用 $\text{numerical_rank}(\dots, 1e-10)$ 与 $\text{numerical_rank}(\dots, 1e-6)$ ，结果应为2与1。再试阈值恰等于第二奇异值的对角例 $\text{diag}(1, 0.5)$ ， $\tau=0.5$ ，结果为1，因为代码使用严格大于。

接着将 $\text{diag}(1, 10^{-8})$ 整体乘 10^6 ，再使用原绝对阈值。说明为什么数值秩判定会受到单位和阈值选择影响，而数学秩不会。可与 $\text{np.linalg.matrix_rank}$ 默认结果比较，但记录NumPy版本；默认阈值不等于你真实测量的误差上限。

任务G 主动触发失败

不能只测试正确输入。请验证下列输入会被明确拒绝：一维数组、空矩阵、布尔、复数、文本、非有限数； k 为负、超过 $\min(\text{shape})$ 、非整数或布尔；阈值为负或非有限；矩形矩阵求逆问题条件数；右端 shape 不为 $(1, n)$ ；奇异矩阵求唯一解；最终计算溢出。

```
from experiment import rank_k
try:
    rank_k([[3., 1.], [1., 3.]], True)
except TypeError as error:
    print(type(error).__name__, str(error))
else:
    raise AssertionError("Boolean rank budget was silently accepted")
```

原始Python容器中的布尔值会在NumPy自动类型提升前拒绝；已被外部代码转成数值dtype的数组不保存来源，函数无法还原。函数只承诺非空二维有限float64输入，不承诺任意动态范围的稳定SVD。返回无穷条件数只用于算出的最小奇异值恰为0；极大有限比例超出float64范围时明确报错。

任务H 独立核验与复现交付

测试脚本用80位Decimal的 2×2 解析奇异值公式核验100个整数矩阵，而不是拿同一SVD函数重复比较。另用Fraction精确分数解28个非对称行系统，避免对称矩阵掩盖转置错误。异常、零矩阵、重复谱、阈值边界、未修改原输入和修改配置后的失败状态也有测试。

将脚本与Notebook分别生成的results.json、rank_errors.csv、perturbations.csv、environment.json比较。相同环境与相同数据下应逐字节一致。跨平台、不同BLAS实现或不同NumPy版本的最后几位浮点数可能不同，不能把本次同环境一致外推为跨平台二进制一致。

交付清单：纸笔SVD、两种截断误差、标签反例、两方向扰动表、残差解释、两个阈值下的数值秩、至少三条失败输入及异常、完整测试输出、环境版本。结论需同时写“证据支持什么”与“仍不能推断什么”。不要求提交额外模型训练或修改系统设置。